Database Management – SQL & PLSQL

Assessment

Section A: Concept Application

1. Explain how relational databases help maintain accuracy in expense records.

Ans.

- Primary key ensures each record is unique
- Foreign Key constraints prevent orphaned records
- Data types (Decimal) prevent invalid data entry
- Transaction management (Commit/Rollback) ensures data integrity
- Constraints (Not Null, Unique) enforce data quality rules

2. Why are constraints important in personal finance data?

Ans.

- Prevent invalid data (negative amounts, duplicate emails)
- Ensure data consistency across related tables
- Protect financial accuracy (critical for budget tracking)
- Reduce errors in reporting and analysis

3. How does GROUP BY help analyze spending patterns?

Ans.

- Aggregates expenses by category, date, or user
- Enables SUM(), AVG(), COUNT() calculations per group
- Shows total spending per category

- Identifies highest/lowest spending categories

4. Explain a scenario where rollback is required during expense entry.

Ans.

Scenario: User enters 5 expense records but discovers incorrect amounts/categories

before Commit. Use Rollback to undo all 5 entries and re-enter correctly.

5. How do views help users track monthly expenses efficiently?

Ans.

- Predefine complex Join queries (simplify repeated queries)
- Hide unnecessary columns
- Provide consistent interface for monthly reports
- Improve query performance

6. Why use triggers for automatic category or balance updates?

Ans.

- Automate data entry (auto-categorize based on amount)
- Maintain real-time balance calculations
- Ensure consistency (no manual update needed)
- Prevent human errors

Section B: SQL Hands-On

Given Database Schema

```
users (
user_id INT PRIMARY KEY,
name VARCHAR(50),
email VARCHAR(100),
created_at DATE
);

categories (
category_id INT PRIMARY KEY,
category_name VARCHAR(50)
);

expenses (
expense_id INT PRIMARY KEY,
user_id INT,
category_id INT,
amount DECIMAL(10,2),
expense_date DATE,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (category_id) REFERENCES categories
(category_id)
);
```

1. DDL Understanding

Without modifying the schema:

- **Explain why foreign keys are used: Specifically, why must user_id in the expenses table correspond to a real ID in the users table?**

Ans.

Foreign keys ensure data integrity by preventing expenses from being inserted for users that don't exist. They maintain the relationship between users and expenses tables.

- **Mention one issue that would occur if foreign keys were removed: Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted).**

Ans.

Without foreign keys, "Orphaned Records" can occur - when a user is deleted but their expenses remain with invalid user_id, making data meaningless.

2. DML Operations

Write SQL queries to:

● Insert 5 users into the users table with unique names and emails.

Ans.

Insert into users (user_id, name, email, created_at) **Values**

(1, 'Rahul Sharma', 'rahul.sharma@email.com', '2025-01-15'),

(2, 'Priya Patel', 'priya.patel@email.com', '2025-02-20'),

(3, 'Amit Kumar', 'amit.kumar@email.com', '2025-03-10'),

(4, 'Sneha Reddy', 'sneha.reddy@email.com', '2025-04-05'),

(5, 'Vikram Singh', 'vikram.singh@email.com', '2025-05-12');

● Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').

Ans.

Insert into categories (category_id, category_name) **Values**

(1, 'Food'),
(2, 'Rent'),

(3, 'Entertainment');

● Insert 10 expense records distributed across different users and categories.

Ans.

Insert into expenses (expense_id, user_id, category_id, amount, expense_date) **Values**

(1, 1, 1, 250.00, '2025-06-01'), -- Rahul, Food

(2, 1, 3, 500.00, '2025-06-02'), -- Rahul, Entertainment

(3, 2, 1, 180.00, '2025-06-01'), -- Priya, Food

(4, 2, 2, 1200.00, '2025-06-01'), -- Priya, Rent

(5, 3, 1, 320.00, '2025-06-03'), -- Amit, Food

(6, 3, 3, 750.00, '2025-06-04'), -- Amit, Entertainment

(7, 4, 1, 200.00, '2025-06-02'), -- Sneha, Food

(8, 4, 2, 1100.00, '2025-06-01'), -- Sneha, Rent

(9, 5, 3, 450.00, '2025-06-05'), -- Vikram, Entertainment

(10, 5, 1, 290.00, '2025-06-03'); -- Vikram, Food

● Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense_id.

Ans.

Update expenses Set amount = 300.00 WHERE expense_id = 1;

-- Verify the update

SELECT * FROM expenses WHERE expense_id = 1;

● Delete one expense: Remove a record where the amount is less than a specific value.

Ans.

Delete From expenses where amount < 200;

-- Verify the deletion

Select * from expenses;

3. Data Retrieval

Write queries to:

- **Display all expenses with details: Show the expense_date, amount, name (from users), and category_name (from categories) using INNER JOIN.**

Ans.

SELECT

e.expense_date,

e.amount,

u.name AS user_name,

c.category_name

From expenses e

Inner Join users u ON e.user_id = u.user_id

Inner Join categories c ON e.category_id = c.category_id

Order by e.expense_date;

- **Show total expense amount per category: Use SUM(amount) and GROUP BY category_name.**

Ans.

Select

c.category_name,

Sum(e.amount) AS total_amount

From expenses e

Inner Join categories c ON e.category_id = c.category_id

Group by c.category_name

Order by total_amount DESC;

- **Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest.**

Ans.

Select

u.name AS user_name,

Sum(e.amount) AS total_spending

From users u

Inner Join expenses e ON u.user_id = e.user_id

Group by u.user_id, u.name

Order by total_spending DESC;

4. Views

- **Create a view named ActiveUsersView: This view should list the name and email of any user who has logged more than 5 individual expense records.**

Ans.

Insert into expenses (expense_id, user_id, category_id, amount, expense_date)

VALUES

(11, 1, 1, 150.00, '2025-06-06'),

(12, 1, 1, 175.00, '2025-06-07'),

(13, 1, 3, 300.00, '2025-06-08'),

(14, 1, 1, 220.00, '2025-06-09'),

(15, 1, 2, 1200.00, '2025-06-10'),

(16, 1, 1, 190.00, '2025-06-11');

-- Verify Rahul now has more than 5 expenses

Select

u.name,

Count(e.expense_id) AS expense_count

From users u

Inner Join expenses e ON u.user_id = e.user_id

Where u.user_id = 1

Group by u.user_id, u.name;

-- Create a view named ActiveUsersView

```
Create View ActiveUsersView AS
Select
u.name,
u.email
From users u
Inner Join expenses e ON u.user_id = e.user_id
Group by u.user_id, u.name, u.email
Having Count(e.expense_id) > 5;
```

- **Query the view: Write a simple SELECT statement to show all data from ActiveUsersView.**

Ans.

-- Query the view: Show all data from ActiveUsersView

```
Select * From ActiveUsersView;
```

Section C: Mini Project

Mini Project: Expense Tracker DB

Using the same schema:

CREATE TABLE USERS:-1

```
Create Table users (
user_id int primary key,
name varchar(50),
email varchar(100),
created_at date
);
```

CREATE TABLE CATEGORIES:-2

```
CREATE TABLE categories (
category_id int primary key,
category_name varchar(50)
);
```

CREATE TABLE EXPENSES:-3

```
Create table expenses (
expense_id int primary key,
user_id int,
category_id int,
amount decimal(10,2),
expense_date date,
```

Foreign key (user_id) REFERENCES users(user_id),

Foreign key (category_id) REFERENCES categories(category_id));

INSERT QUERY USERS:-1

Insert into users (user_id, name, email, created_at) Values

(1, 'Rahul Sharma', 'rahul.sharma@email.com', '2025-01-15'),

(2, 'Priya Patel', 'priya.patel@email.com', '2025-02-20'),

(3, 'Amit Kumar', 'amit.kumar@email.com', '2025-03-10');

INSERT QUERY CATEGORY :-2

Insert into categories (category_id, category_name) Values

(1, 'Food'),

(2, 'Rent'),

(3, 'Entertainment');

INSERT QUERY EXPENSES:-3

Insert into expenses (expense_id, user_id, category_id, amount, expense_date) Values

(1, 1, 1, 250.00, '2025-06-01'),

(2, 1, 3, 500.00, '2025-06-02'),

(3, 2, 1, 180.00, '2025-06-01'),

(4, 2, 2, 1200.00, '2025-06-01'),

(5, 3, 1, 320.00, '2025-06-03');

- **Write CRUD queries**

Ans.

-- Create: Insert new expense

Insert into expenses (expense_id, user_id, category_id, amount, expense_date) VALUES (6, 1, 2, 1100.00, '2025-06-10');

-- Verify Create

Select '=== Create: New expense inserted ===' AS operation;

Select * From expenses WHERE expense_id = 6;

-- Read*:* Select all expenses

SELECT '=== READ: All expenses ===' AS operation;

Select * From expenses;

-- Read: Select specific user's expenses

Select '=== Read: User 1 expenses ===' AS operation;

Select * From expenses Where user_id = 1;

-- Update: Modify existing expense

Update expenses

Set amount = 1200.00,

    expense_date = '2025-06-11'

Where expense_id = 6;

-- Verify Update

Select '=== Upadet: Expense modified ===' AS operation;

Select * From expenses Where expense_id = 6;

-- Delete: Remove expense

Delete From expenses

Where expense_id = 6;

-- Verify DELETE

Select '=== Delete: Expense removed ===' AS operation;

Select * From expenses Where expense_id = 6;

- **Write a stored procedure to calculate monthly user expense**

Ans.

-- Drop procedure if exists (avoid error*)*

Drop Procedure if exists CalculateMonthlyExpense;

-- Create stored procedure

Create Procedure CalculateMonthlyExpense(

In p_user_id Int,

In p_month Int,

In p_year Int

)

Begin

```
-- Select monthly expense breakdown
Select
    u.name AS user_name,
    c.category_name,
    Sum(e.amount) AS monthly_total,
    Count(e.expense_id) AS expense_count,
     Avg(e.amount) AS average_expense
From expenses e
Inner Join users u ON e.user_id = u.user_id
Inner Join categories c ON e.category_id = c.category_id
Where e.user_id = p_user_id
    And Month(e.expense_date) = p_month
    And Year(e.expense_date) = p_year
Group by c.category_name
Order by monthly_total DESC;

-- Show total monthly expense summary
Select Concat(
      'Total Monthly Expense for User ', p_user_id, '
       (Month ', p_month, ', Year ', p_year, '): ',
Format(Sum(e.amount), 2)
) AS total_summary
From expenses e
Where e.user_id = p_user_id
Aan Month(e.expense_date) = p_month
```

```
And Year(e.expense_date) = p_year;
End;
```

-- Call the procedure - Example 1: User 1, June 2025

```
Select '=== C2: Stored Procedure - User 1, June 2025 ===' AS operation;
Call CalculateMonthlyExpense(1, 6, 2025);
```

-- Call the procedure - Example 2: User 2, June 2025

```
Select '=== C2: Stored Procedure - User 2, June 2025 ===' AS operation;
Call CalculateMonthlyExpense(2, 6, 2025);
```

-- Call the procedure - Example 3: User 3, June 2025

```
Select '=== C2: Stored Procedure - User 3, June 2025 ===' AS operation;
Call CalculateMonthlyExpense(3, 6, 2025);
```

- **Demonstrate COMMIT and ROLLBACK with example queries**

Ans.

-- Rollback Example 1: Undo incorrect batch data entry

```
SELECT '=== C3: ROLLBACK Example 1 - Undo incorrect expenses ===' AS operation;
```

-- Start transaction

Start TransAaction;

-- Insert multiple expense records

Insert Into expenses (expense_id, user_id, category_id, amount, expense_date) Values

(7, 3, 1, 280.00, '2025-06-15'),

(8, 3, 3, 600.00, '2025-06-15'),

(9, 1, 1, 210.00, '2025-06-15');

-- Verify insertions

Select 'Step 1: After INSERT - Records in transaction:' AS step;
Select expense_id, user_id, amount From expenses Where expense_id IN (7, 8, 9);

-- Rollback: Undo all insertions

Rollback;

-- Verify rollback

Select 'Step 2: After Rollback - Records should Not exist:' AS step;

SELECT expense_id, user_id, amount From expenses Where expense_id IN (7, 8, 9);

-- Commit Example 2: Save correct data entry

Select '=== C3: Commit Example 2 - Save correct

expenses ===' AS operation;

-- Start new transaction

Start Transaction;

-- Insert correct expense records

Insert Into expenses (expense_id, user_id, category_id, amount, expense_date) Values

(7, 3, 1, 280.00, '2025-06-15'),

(8, 3, 3, 600.00, '2025-06-15'),

(9, 1, 1, 210.00, '2025-06-15');

-- Verify insertions

Select 'Step 1: After INSERT - Records ready to commit:'

AS step;

Select expense_id, user_id, amount From expenses Where expense_id IN (7, 8, 9);

-- COMMIT: Save all changes permanently

Commit;

-- Verify commit

Select 'Step 2: After COMMIT - Records saved permanently:' AS step;

Select expense_id, user_id, amount From expenses Where expense_id IN (7, 8, 9);

```
-- Rollback Example 3: Update with Rollback
Select '=== C3: Rollback Example 3 - Undo amount
change ===' AS operation;

-- Start transaction
Start Transaction;

-- Update expense amount
Update expenses
Set amount = 500.00
Where expense_id = 7;

-- Verify update
Select 'Step 1: After Update - Amount changed to 500:' AS step;
Select expense_id, amount From expenses Where
expense_id = 7;

-- Rollback: Undo the update
Rollback;

-- Verify rollback
Select 'Step 2: After ROLLBACK - Amount back to 280:'
AS step;
Select expense_id, amount FROM expenses Where
expense_id = 7;
```

-- Commit Example 4: Update with Commit

```
Select '=== C3: Commit Example 4 - Save amount
change ===' AS operation;

-- Start Transaction
Start Transaction

-- Update expense amount
Update expenses
Set amount = 350.00
Where expense_id = 7;

-- Verify update
Select 'Step 1: After UPDATE - Amount changed to 350:'
    AS step;
Select expense_id, amount From expenses Where expense_id = 7;

-- Commit: Save the update permanently
Commit;

-- Verify commit
Select 'Step 2: After Commit - Amount saved as 350:' AS step;
Select expense_id, amount FROM expenses
   Where expense_id = 7;
```

-- Final Verification: Display all data

```
SELECT '========================================' AS '';
SELECT 'FINAL VERIFICATION - All Data:' AS '';
SELECT '========================================' AS '';

Select 'Users Table' AS section;
Select * From users;

Select 'Categories Table' AS section;
Select * From categories;

Select ' Expenses Table' AS section;
Select * From expenses;
```

-- Call stored procedure for final verification

```
Select 'Monthly Expense Summary - User 1, June 2025:'
   AS section;
Call CalculateMonthlyExpense(1, 6, 2025);
```

--
==========

-- END OF SECTION C - MINI PROJECT --

==========
===